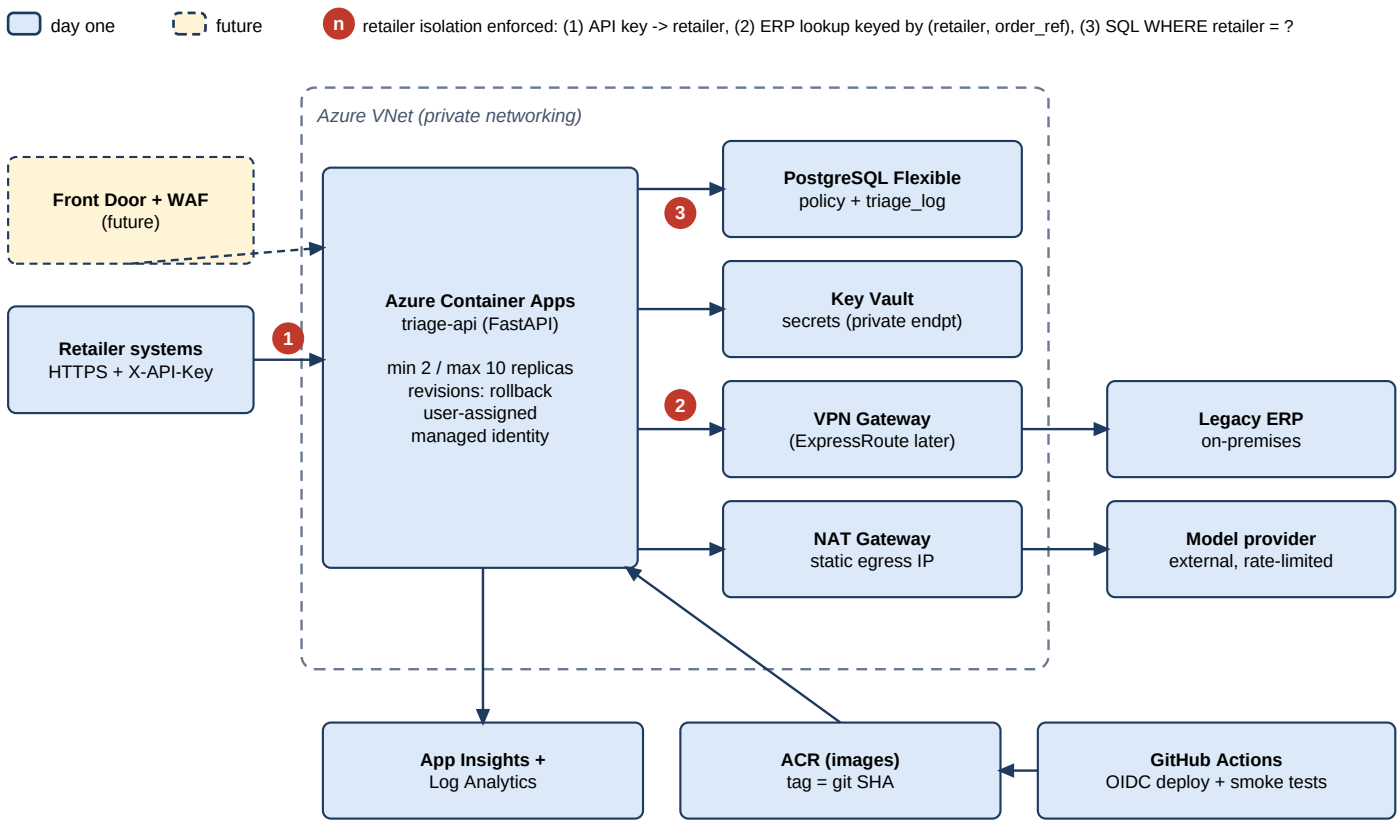
Azure Plan: Support Triage Service

Status: design only. Nothing was deployed, and nothing here was tested against an Azure subscription. Placeholders are written as <LIKE_THIS>. The plan describes the **target** system; the local code uses SQLite and a file-based ERP.

1. Assumptions

- 2 retailers growing to 100. 10,000 tickets/day averages ~0.12 req/s, but bursts reach **20 req/s**.
- External model provider with rate limits and timeouts; worst-case model call 8 s.
- Legacy ERP is on-premises and reachable only over private networking, with a limited connection count.
- Customer messages may be sensitive: they are never stored or logged (the audit log keeps metadata only).
- Single region, ~99.9% target, small team: managed services only, no Kubernetes. Triage is synchronous today; async is a future option.

2. Architecture and service selection



Service	Purpose	Why it fits
Azure Container Apps	Runs the FastAPI container	Scales on concurrent requests; revisions give instant rollback; no cluster to manage for a small team.
PostgreSQL Flexible Server	policy + triage_log (relational store)	My SQL already uses ON CONFLICT upserts (Postgres-compatible; Azure SQL needs MERGE). Built-in PgBouncer, PITR backups, Entra auth.
VPN Gateway	Private path to on-prem ERP	ERP must not be public. VPN is the cheaper day-one option; ExpressRoute later. The ErpClient interface hides the transport.
Key Vault	Model key, ERP credential, API keys	Central rotation and audit; read through managed identity.
ACR / App Insights + Log Analytics / NAT Gateway	Images / telemetry and alerts / static outbound IP	Image tagged by git SHA = exact rollback; one place for incident queries; provider can allow-list the egress IP.

How the app reaches the ERP: VNet integration plus a site-to-site VPN to the ERP network. Only the adapter implementation changes; the service code stays the same. **Alternative considered:** Azure App Service. Simpler and familiar, but weaker revision/traffic-split and concurrency-based scaling; AKS rejected as too much operations work for one small service.

Day one	Future additions
Container Apps (min 2), PostgreSQL (burstable, PITR), Key Vault, ACR, App Insights, VPN, NAT, GitHub Actions	Front Door + WAF, APIM or Entra OAuth for 100 retailers, Postgres Row-Level Security, zone-redundant HA, Service Bus async path, ExpressRoute

3. Identity, secrets and access

Callers: day one, API keys held in Key Vault and mapped to a retailer inside the app; each retailer has **two active keys** so keys rotate with overlap. For 100 retailers: OAuth client credentials with a retailer claim, or APIM subscriptions. Retailer identity always comes from the key, never from the message or order_ref.

Identity	Resource	Permission	Scope
id-triage-api (user-assigned managed identity)	Key Vault	Key Vault Secrets User	That one vault
id-triage-api	ACR	AcrPull	That registry
id-triage-api	PostgreSQL (Entra auth)	DB role triage_app: SELECT on policy, INSERT on triage_log only	That database
id-triage-migrate (used by migration job only)	PostgreSQL	DDL rights for schema/seed migration	That database
GitHub Actions (OIDC federated credential)	Resource group, ACR	Contributor, AcrPush	That resource group

- **No stored cloud passwords.** The app cannot alter or delete audit rows (INSERT only).
- **Model key rotation (~90 days):** add a new secret version, keep the old key valid during overlap, switch, then revoke the old one; alert before expiry. If the provider is Azure OpenAI, use managed identity (Cognitive Services OpenAI User role) and no key at all.
- **ERP credential:** Key Vault secret; the ERP-side account is read-only on an orders view.
- **Log protection:** never log message bodies or keys (code logs request_id/ticket_id only); Log Analytics readable by a team group only; 30-90 day retention; TLS to Postgres.

4. Deployment and rollback

- **PR checks:** pytest, the evaluation command, gitleaks secret scan.
- **Build:** container image tagged with the git SHA, pushed to ACR (OIDC login, no stored secrets).
- **Migration:** additive-only (expand/contract) schema changes run by a migration job as id-triage-migrate; the policy seed is an idempotent upsert, safe to rerun.
- **Deploy:** new Container Apps revision at 0% traffic; readiness probe on /health.
- **Verify:** smoke script sends known tickets and checks the decisions and the citations.
- **Gate and shift:** manual approval on the production environment, then 10%, then 100% traffic.
- **Rollback:** shift traffic back to the previous revision (instant, no redeploy). Safe because migrations were additive.

Illustrative workflow (UNTESTED, replace placeholders):

```
name: deploy
on: { push: { branches: [main] } }
permissions: { id-token: write, contents: read }          # OIDC: no stored cloud secrets
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install -r requirements.txt
      - run: python -m pytest -q && python -m tests.run_eval
  deploy:
    needs: test
    runs-on: ubuntu-latest
    environment: production                                 # required reviewers = release gate
    steps:
      - uses: actions/checkout@v4
      - uses: azure/login@v2
        with: { client-id: "${{ vars.AZURE_CLIENT_ID }}", tenant-id: "<TENANT>", subscription-id: "<SUB>" }
      - run: az acr build -r <ACR> -t triage:${{ github.sha }} .
      - run: az containerapp job start -n triage-migrate -g <RG>
      - run: |
          az containerapp update -n triage-api -g <RG> \
            --image <ACR>.azurecr.io/triage:${{ github.sha }} --revision-suffix g${GITHUB_SHA::7}
      - run: ./scripts/smoke.sh https://<new-revision-fqdn>
      - run: |
          az containerapp ingress traffic set -n triage-api -g <RG> \
            --revision-weight g${GITHUB_SHA::7}=100
```

Illustrative app configuration (UNTESTED):

```
properties:
  configuration:
    activeRevisionsMode: Multiple
    ingress: { external: true, targetPort: 8000 }
    secrets:
      - { name: model-api-key, keyVaultUrl: "https://<VAULT>.vault.azure.net/secrets/model-api-key", identity: "<UAMI-RESOURCE-ID>" }
  template:
    containers:
      - name: triage-api
        image: <ACR>.azurecr.io/triage:<SHA>
        env: [ { name: MODEL_PROVIDER, value: http }, { name: LLM_API_KEY, secretRef: model-api-key } ]
        probes: [ { type: Readiness, httpGet: { path: /health, port: 8000 } } ]
    scale: { minReplicas: 2, maxReplicas: 10, rules: [ { name: http, http: { metadata: { concurrentRequests: "30" } } } ] }
```

How I would validate with Azure access: deploy to a throwaway resource group and run `az deployment group what-if`; ship a deliberately broken revision and confirm traffic never shifts; remove the Key Vault role and confirm behaviour matches incident A; rehearse rollback; run a load test (k6 or Azure Load Testing).

5. Scaling and provider limits

Requests in flight (Little's law): 20 req/s x 8 s = **160 concurrent model-bound requests** at burst. This is the worst case: only policy_supported and needs_information tickets call the model; human_review replies do not.

Known gap in my code: the service caps concurrent ERP/model calls with a 16-thread pool. At 8 s per call that is ~2 req/s per replica, far below 20. Before launch I would move those calls to async I/O (httpx.AsyncClient) so 160 in-flight calls are cheap, then scale on concurrency.

Concern	Plan
Replicas	~30 concurrent requests per replica gives ~6 replicas at burst; max 10. Keep min 2 because scale-out takes time (unproven without a load test).
Timeouts	ERP 2 s, model 8 s, overall deadline ~10 s; client timeout set longer than that.
Retries	At most one retry on 429/5xx with jitter, honouring Retry-After, only if time remains. No retry after a timeout (doubles latency).
Provider quota	Token-bucket rate limiter plus a concurrency semaphore (bulkhead) around model calls, set below the provider's requests/min and tokens/min.
ERP limits	replicas x pool size must stay under the ERP's connection limit; circuit breaker so failures are fast.
Database	~20 inserts/s is small; use pooling (PgBouncer) and check max connections against replica count.
Overload	Shed load with 503 + Retry-After instead of unbounded queuing.

- **Today (sync):** dependency failure returns 200 with human_review and model.fallback=true; overload returns 503 with Retry-After.
- **Future async:** POST returns 202 with a request_id; a Service Bus worker processes it; the client polls GET /triage/{request_id} or receives a webhook.

6. Cost and remaining uncertainty

Grows with traffic	Paid while idle
Model tokens (dominant cost); Container Apps compute above the minimum replicas; egress; Log Analytics ingestion.	PostgreSQL (always on, plus storage/backups); minimum replicas; VPN Gateway or ExpressRoute (hourly); NAT Gateway; Log Analytics retention; ACR.

- **Limiting waste:** no model call for human_review (already done); max_tokens 150, temperature 0; log IDs and counts, not bodies; 30-day hot logs then archive; scale dev to zero and stop dev Postgres at night; resource tags, budget alerts, and a **cost per ticket** metric.
- **Unproven without deployment and load testing:** real model p95 and quota behaviour; ERP latency over the VPN; Container Apps scale-out speed at a 20 req/s burst; failover times; Key Vault throttling; actual cost.

7. Incidents (first three checks, in order)

A. API starts successfully, but Key Vault access fails	
First three checks	**1.** Key Vault `AuditEvent` diagnostic logs: filter SecretGet for the app identity's object ID and read the result code. **2.** Identity and recent changes: `az role assignment list --assignee <principalId> --scope <vault>`; Activity Log for deleted roles, firewall changes, disabled/expired secrets. **3.** Network from the container: `az containerapp exec`, resolve <vault>.vault.azure.net; it should return a private IP.
Telling causes apart	ForbiddenByRbac = missing role. ForbiddenByFirewall = network rule. Token error = identity not attached. Timeouts with no audit entries = DNS or private endpoint. 429 = throttling. SecretDisabled/not found = secret disabled or removed.
Immediate mitigation	Restore the role or rule. App keeps last-known-good secrets in memory and refreshes with backoff so running replicas keep serving. Never open the vault publicly as a fix.
Verify recovery	Audit log shows successful SecretGet; /health OK; smoke test passes; no 5xx in the requests table.

B. After a release, latency rises sharply and some requests time out	
First three checks	**1.** Compare by revision: App Insights `requests | summarize percentiles(duration,50,95,99) by cloud_RoleInstance`; line up onset with the deploy time. **2.** Dependency breakdown: `dependencies` table duration by target (model, ERP, Postgres), before vs after. **3.** Saturation: replica count, CPU/memory, concurrent-requests metric, restarts or OOM in system logs.
Telling causes apart	Only the new revision slow = code/config regression (timeouts, pool size, new query). Both revisions slow = dependency. Slow right after scaling = cold start. Dependency time flat but app time up = CPU or thread starvation.
Immediate mitigation	Shift traffic back: `az containerapp ingress traffic set --revision-weight <prev>=100`.
Verify recovery	p95 back under target for 15-30 minutes; no timeouts; fallback rate in triage_log at baseline.

C. Model provider returns intermittent rate-limit (429) errors	
First three checks	**1.** Quantify: 429 count vs total model calls per minute, whether Retry-After is present, and the fallback=1 rate in triage_log. **2.** Compare to quota: our request rate and concurrency (replicas x per-replica concurrency) vs the provider's requests/min and tokens/min dashboard. **3.** Recent changes: a scale-out, a release that changed max_tokens or retries (retry storms amplify 429s), another consumer sharing the key, provider status page.
Telling causes apart	429s aligned with our bursts = we exceed quota. 429s at low volume = shared quota or provider-side throttling. The error body says whether the limit is requests/min or tokens/min.
Immediate mitigation	Lower concurrency (semaphore or replica cap), back off honouring Retry-After, request more quota or add a second deployment. Degraded mode only if the business approves.
Verify recovery	429 rate near zero; fallback rate back to baseline; quota use below ~80%.

D. Triage requests return human_review far more often; audit log shows a spike in ERP timeouts	
First three checks	**1.** Confirm it is the ERP: in triage_log group decision and sum(fallback) by hour. Fallback jumps while the category mix is stable = a dependency, not classification. (Schema has only a boolean fallback; I would add a fallback_reason column. Until then compare 'ERP lookup failed' vs 'model draft failed' log counts.) **2.** ERP dependency telemetry: failures by target; timeouts at exactly our 2 s limit vs fast failures (refused, DNS, auth); VPN tunnel state and DNS resolution of the ERP host. **3.** ERP side: ask the ERP team about load, batch jobs, maintenance and connection limits; did our scale-out exceed their limit?
Telling causes apart	Timeouts at 2 s = ERP slow or unreachable. Fast failures = network, DNS, auth or connection limit. Spike after scale-out = connection exhaustion.
Immediate mitigation	Circuit breaker so requests fail fast; cap replicas or pool size. Order facts are never fabricated, so tickets stay in human review; alert on backlog and reprocess after recovery.
Verify recovery	ERP timeout rate near zero; human_review share back to normal; latest rows have fallback=0; backlog reprocessed.